

Corridor.

An emergency-routing atlas for Bengaluru

Requirements and Design Document



This document states what the Corridor prototype is required to do, describes how it is built module by module, and records the evidence that it works: ten executable tests and a benchmark on eight city sizes. Every screenshot was taken from the running prototype and every number was read from it on the date below.

PREPARED BY	Nebin Stanly
REGISTER NUMBER	2443142
CLASS	6BCA-B, Computer Science (Yeshwanthpur)
COURSE	Design and Analysis of Algorithms
STAFF-IN-CHARGE	Dr. P. Margaret Savitha
DOCUMENT VERSION	1.0, 9 October 2026
LIVE PROTOTYPE	https://h0gr1d3r-code.github.io/corridor-routing-atlas/
SOURCE CODE	https://github.com/H0GR1D3R-CODE/corridor-routing-atlas

CONTENTS

1	Introduction	<i>purpose, problem statement, scope, definitions</i>
2	Overall description	<i>users, use cases, environment, constraints</i>
3	System architecture	<i>layers, data flow of one request, data model</i>
4	Functional requirements	<i>twenty-four numbered requirements</i>
5	Module descriptions	<i>every screen, with annotated screenshots</i>
6	Algorithms and design justification	<i>the five searches, the heuristic, complexity</i>
7	Non-functional requirements	<i>performance, portability, accessibility</i>
8	Verification and results	<i>tests, stories, scenes, benchmarks</i>
9	Deployment and operation	<i>where it runs and how to run it</i>
10	Limitations and future work	
11	Traceability to the assignment rubric	

AT A GLANCE

MEASURE	VALUE	MEASURE	VALUE
Junctions (graph nodes)	832	Algorithms implemented	5, plus hospital dispatch
Road segments (edges)	1,630	Executable tests	10 of 10 passing
Named places	62	Benchmark city sizes	8 (from 64 to 8,011 junctions)
Hospitals	15	Scene controls	5 day types, 4 skies, 8 events
Named highways and ring roads	11 + 4	Third-party libraries	None

1 Introduction

1.1 Purpose

Corridor is a working prototype built for the CIA 3 Innovative Assignment, which asks for an algorithm applied to a practical routing problem, with performance analysis and a justification of the algorithm chosen. This document is its requirements and design record. It is written so that an examiner can check each requirement against a screen, a module and a test.

1.2 Problem statement

When an ambulance is called, the dispatcher needs two answers within a second: **which hospital**, and **which roads**. Neither can be read off a printed map, because the time a road takes depends on the hour, the weather and what else is happening in the city. The shortest road is often not the fastest.

In algorithm terms this is a shortest-path problem on a weighted graph whose edge weights are a function of the current scene, queried repeatedly, with edges that can disappear when a road floods or is closed.

ITEM	DEFINITION
Input	A road graph of 832 junctions and 1,630 road segments; an origin; either a destination or the set of 15 hospitals; the scene (day type, hour, sky, events); siren on or off; the set of closed roads.
Output	The route with the smallest total travel time, its ETA, distance, mean speed and the named roads it uses. For dispatch, every hospital ranked by ETA.
Constraints	Travel times are never negative. The answer must be provably optimal, because a wrong route is worse than a slow one. Each query must finish in milliseconds.
Chosen algorithms	A* with an adaptive admissible heuristic for a trip from A to B. Multi-target Dijkstra for "nearest hospital". Dijkstra, bidirectional Dijkstra, Bellman–Ford and greedy best-first are implemented as comparisons.

1.3 Scope

IN SCOPE	OUT OF SCOPE
Fastest route between any two junctions	Turn-by-turn navigation or voice guidance
Fastest hospital from an incident, with a ranked list	Live traffic feeds: traffic is a model, not measured data
A scene model: day type, hour, weather, events, siren, closures	Real street geometry: the mesh between real places is generated
Visual explanation: search replay, race, step-through	Fleet scheduling across several ambulances
Built-in correctness tests and benchmarks	User accounts, storage or any server component

1.4 Definitions

TERM	MEANING IN THIS DOCUMENT
Junction	A node of the road graph: a place where roads meet.
Road segment	An edge of the graph, joining two junctions. All segments are two-way.
Weight	The minutes needed to travel one road segment in the current scene.
Scene	The conditions that set the weights: day type, hour, sky, events, siren and closed roads.
ETA	Estimated time of arrival: the total weight of the route, in minutes.

TERM	MEANING IN THIS DOCUMENT
Expanded (settled)	A junction whose shortest time is final. The count of these measures how far a search spread.
Edge examined (relaxation)	One check of a road to see whether it gives a faster way to its far end. The unit of work used for comparisons.
Heuristic h	A quick estimate of the time still needed to reach the goal. Admissible means it never over-estimates.
CSR	Compressed sparse row: the adjacency lists of all junctions packed into flat arrays.
Dispatch	Finding the fastest hospital for an incident, without knowing the destination in advance.

2 Overall description

2.1 Product perspective

Corridor is a single web page. It needs no installation, no build step and no server: the road graph is generated in the browser when the page loads, and every search, test and benchmark runs on the viewer's own machine. The only network requests are for two typefaces, and the page falls back to system fonts if they are unavailable.

2.2 Users and use cases

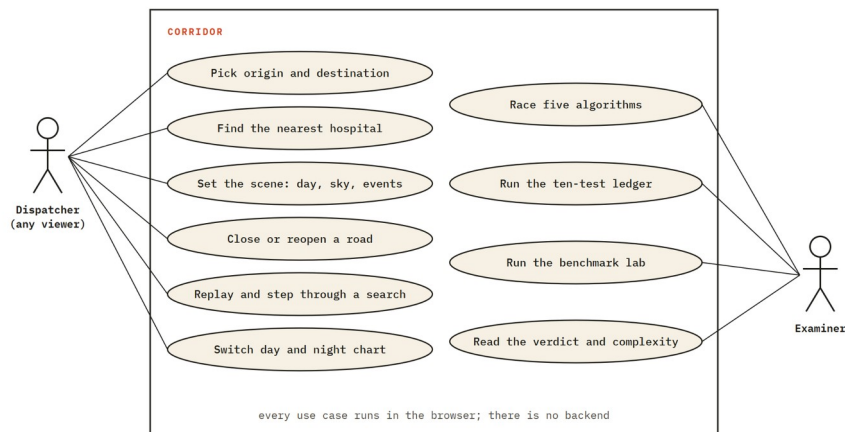


Figure 1. Use cases. A viewer acts as dispatcher; an examiner additionally runs the race, the tests and the benchmark.

ID	USE CASE	WHAT THE USER DOES	WHAT THE SYSTEM RETURNS
UC-1	Route A to B	Picks two places from the lists or clicks the map	Fastest route, ETA, roads used, search statistics
UC-2	Dispatch	Marks an incident and leaves the destination on "Nearest hospital"	Fastest hospital, route to it, all hospitals ranked
UC-3	Change the scene	Taps a preset or sets day, hour, sky and events	New road times, re-planned route, slowdown explained
UC-4	Close a road	Shift-clicks a road on the map	Road shown closed; route re-planned around it
UC-5	Understand a search	Presses Replay or Big picture	The search replayed step by step, with live counters
UC-6	Compare algorithms	Runs the race	Five searches on one trip, ranked by work done
UC-7	Verify correctness	Scrolls to the proof ledger	Ten tests run live, each with its evidence
UC-8	Measure performance	Scrolls to the lab	Timings and work on eight city sizes, as charts and a table

2.3 Operating environment

- Any current desktop or mobile browser (Chrome, Edge, Firefox, Safari). No plug-ins.
- Screens from 375 px wide phones to wide desktop monitors.
- Works offline once loaded; opened directly from disk or from any static web host.

2.4 Assumptions and constraints

- The 62 places, 15 hospitals, lakes and highway bearings are real and positioned from approximate latitude and longitude. The street mesh between them is generated.
- Free-flow speeds are assumed: 28 km/h on local streets, 45 km/h on ring roads, 55 km/h on highways.
- The strength of each weather and event effect, and the siren factor of 0.45, are assumptions, not measurements.
- Roads are two-way; signals, one-way streets and turn delays are not modelled.

3 System architecture

3.1 Layers and modules

The code is split so that the routing engine knows nothing about the page. The engine files can be loaded in Node and tested without a browser, which is how the tests were first run. One controller file connects the controls to the engine and tells the drawing modules what to show.

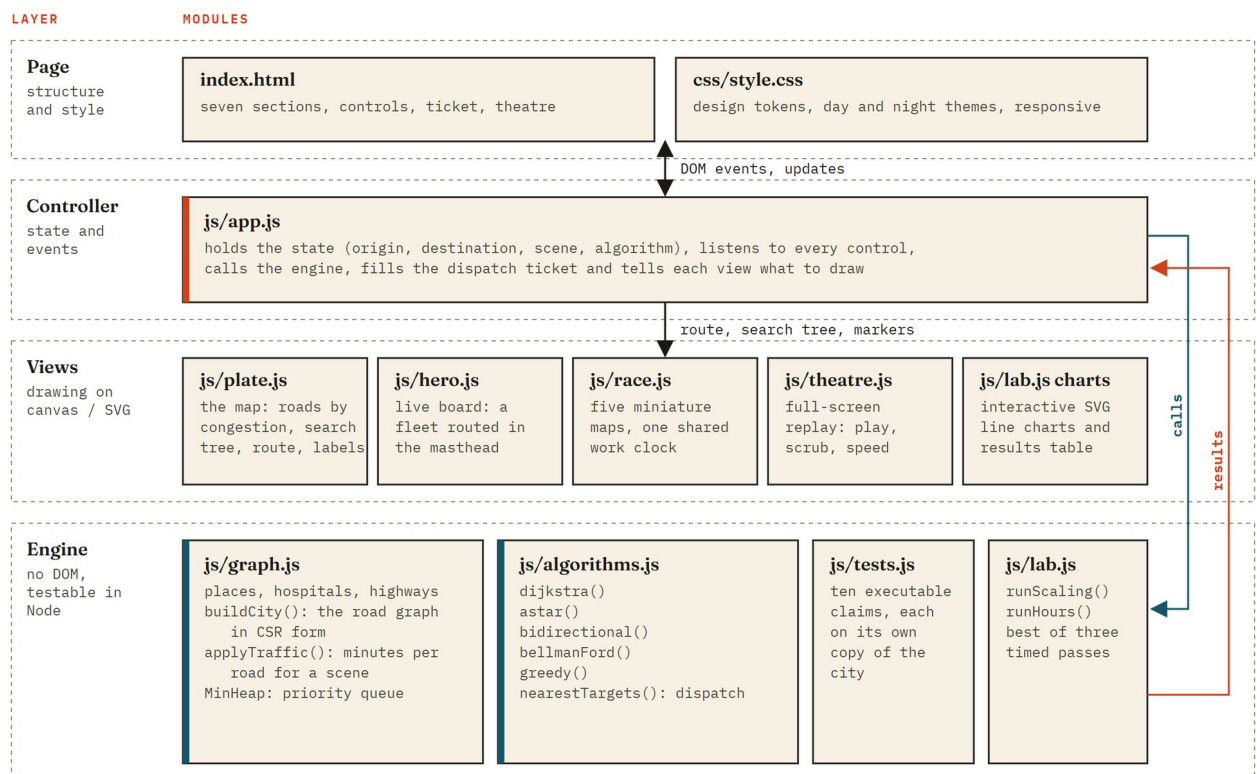


Figure 2. Architecture. The controller calls the engine and passes its results to the views; the engine has no dependency on the page.

FILE	LINES	LAYER	RESPONSIBILITY
index.html	431	Page	The seven sections, the controls, the dispatch ticket and the full-screen replay markup.

FILE	LINES	LAYER	RESPONSIBILITY
css/style.css	493	Page	Colour and type tokens, day and night themes, responsive layout.
js/app.js	621	Controller	Application state; place pickers; scene chips; ticket; replay; wiring of every control.
js/graph.js	569	Engine	Place data, city generation, CSR graph, the traffic model <code>applyTraffic()</code> , the binary heap.
js/algorithms.js	246	Engine	Dijkstra, A*, bidirectional Dijkstra, Bellman–Ford, greedy best-first, multi-target dispatch.
js/tests.js	214	Engine	Ten executable tests, each on its own copy of the city.
js/lab.js	314	Engine + view	The two experiments and the interactive SVG charts.
js/plate.js	347	View	The canvas map: roads, closures, search tree, route, labels, ambulance.
js/hero.js	122	View	The live board in the masthead.
js/race.js	92	View	Five miniature maps driven by one shared work clock.
js/theatre.js	159	View	The full-screen step-through replay.

3.2 Data flow of one request

ONE DISPATCH REQUEST, LEFT TO RIGHT



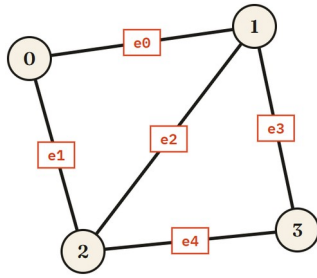
Figure 3. What happens between a click and an ambulance moving on the map.

Because the chain is short and each search takes well under a millisecond on this city, the whole chain is simply re-run whenever anything changes. Dragging the clock re-plans the route continuously.

3.3 Data model

The road network is stored in compressed sparse row form: three flat typed arrays hold every junction's neighbours, and one array per attribute holds the properties of each road. Only the array *W*, the minutes each road takes now, changes when the scene changes.

A SMALL ROAD GRAPH



4 junctions (nodes 0 to 3)
5 road segments (edges e0 to e4)
each road is two-way, so it appears
in the list of both of its ends

HOW IT IS STORED (COMPRESSED SPARSE ROW)

	0	1	2	3	4	5	6	7	8	9
adjStart	0	2	5	8	10	where each node's list begins				
adjTo	1	2	0	2	3	0	1	3	1	2
adjEdge	0	1	0	2	3	1	2	4	3	4

Neighbours of node 1 are adjTo[2..4] = 0, 2, 3, reached by roads adjEdge[2..4] = e0, e2, e3.
One flat array instead of a list per node: fast to scan and friendly to the CPU cache.

PER-ROAD ARRAYS (ONE ENTRY PER EDGE)

eu, ev the two ends	len length, km	cls local, ring, highway
base free-flow minutes	W minutes right now	blocked 0 open, 1 user, 2 scene

THE WEIGHT OF A ROAD

$$W[e] = \text{base}[e] \times (1 + k \times (\text{timeOfDay} + \text{weather} + \text{events}))$$

$k = 1$, or 0.45 with the siren on

Figure 4. The graph data structure, shown on a four-junction example, and the formula that sets each road's weight.

PROPERTY OF THE CITY	VALUE	PROPERTY OF THE CITY	VALUE
Junctions	832	Local streets	28 km/h free flow
Road segments	1,630	Ring roads (4)	45 km/h free flow
Named places	62	Highways (11)	55 km/h free flow
Hospitals	15	Lakes and parks	11
City radius	about 29 km	Zones for the pickers	7

4 Functional requirements

Each requirement is implemented in the prototype. The last column says where it can be seen or which test checks it. Section numbers refer to this document; T-numbers refer to the test ledger in section 8.

ID	REQUIREMENT	MODULE	SHOWN OR VERIFIED
Map and places			
FR-01	Show the road network with every road coloured by its current congestion.	plate.js	\$5.2
FR-02	Offer 62 named places and 15 hospitals through searchable lists grouped by zone.	app.js	\$5.2.1
FR-03	Accept any junction as origin or destination by clicking the map.	app.js, plate.js	\$5.2.2
FR-04	Swap origin and destination in one action.	app.js	\$5.2.1
Routing			
FR-05	Compute the minimum-time route between two junctions and report ETA, distance, mean speed and named roads used.	algorithms.js	T1, T5, T7
FR-06	Find the fastest hospital for an incident and rank all hospitals by ETA.	algorithms.js	T9
FR-07	Report an unreachable destination as unreachable; never return a made-up route.	algorithms.js	T3
FR-08	Provide five interchangeable algorithms on the same graph.	algorithms.js	\$6, T5

ID	REQUIREMENT	MODULE	SHOWN OR VERIFIED
FR-09	Report junctions expanded, edges examined and search time for every search.	algorithms.js	§5.2.4
Scene			
FR-10	Let the user set the day type (5), the time in 15-minute steps, the sky (4) and any of 8 events.	app.js	§5.2.3
FR-11	Recompute every road's travel time for the scene and re-plan the current trip at once.	graph.js	§3.2, T10
FR-12	Close roads automatically in a flood or accident, and let the user close or reopen any road.	graph.js, app.js	T4
FR-13	Model a siren that reduces every delay for the ambulance.	graph.js	T10
FR-14	Explain the slowdown: city-wide by cause, and per trip as "where the minutes go".	app.js	§5.2.3, §5.2.4
FR-15	Offer one-tap scene presets (8) and stories that set a trip and a scene together (6).	app.js	§8.2, §8.3
Explanation			
FR-16	Animate the search in the order junctions were visited, then the route, then an ambulance driving it; replay on demand.	plate.js, app.js	§5.2.2
FR-17	Offer a full-screen replay with play, pause, scrubbing, speed control and a switch between algorithms.	theatre.js	§5.4
FR-18	Race the five algorithms on one trip using a shared clock of edges examined, and rank them.	race.js	§5.3
FR-19	Show a live board on which ambulances are dispatched continuously; clicking one opens that dispatch.	hero.js	§5.1
Verification			
FR-20	Run ten correctness tests in the browser and show each result with its evidence.	tests.js	§5.5, §8.1
FR-21	Benchmark the algorithms on eight city sizes, chart the results interactively and export them as CSV.	lab.js	§5.6, §8.4
FR-22	Show how the advantage of A* changes through the day; clicking an hour sets the console clock.	lab.js	§5.6
Presentation			
FR-23	Provide a day and a night colour theme.	style.css	§5.8
FR-24	Remain usable on a 375 px phone and from the keyboard (pickers and charts).	style.css, app.js	§5.8, §7

5 Module descriptions

The page has seven numbered sections. This chapter walks through each screen in the order a viewer meets it.

5.1 Masthead and live board

The first screen states the question and shows it being answered. The **live board** is a strip of central Bengaluru on which a small fleet is always at work: an incident appears at a random junction, one multi-target Dijkstra

search finds the fastest hospital under the current traffic, and an ambulance drives the route it returned. Each ambulance carries a flag naming its hospital and the minutes left. Clicking one opens that dispatch in the console. Nothing on the board is scripted; it follows whatever scene the console is set to.



Figure 5. Masthead: title, problem in one sentence, the live board with three dispatches in progress, key figures and section navigation.

5.2 Dispatch console

The console is the prototype proper. Everything the assignment calls "the routing solution" happens here.

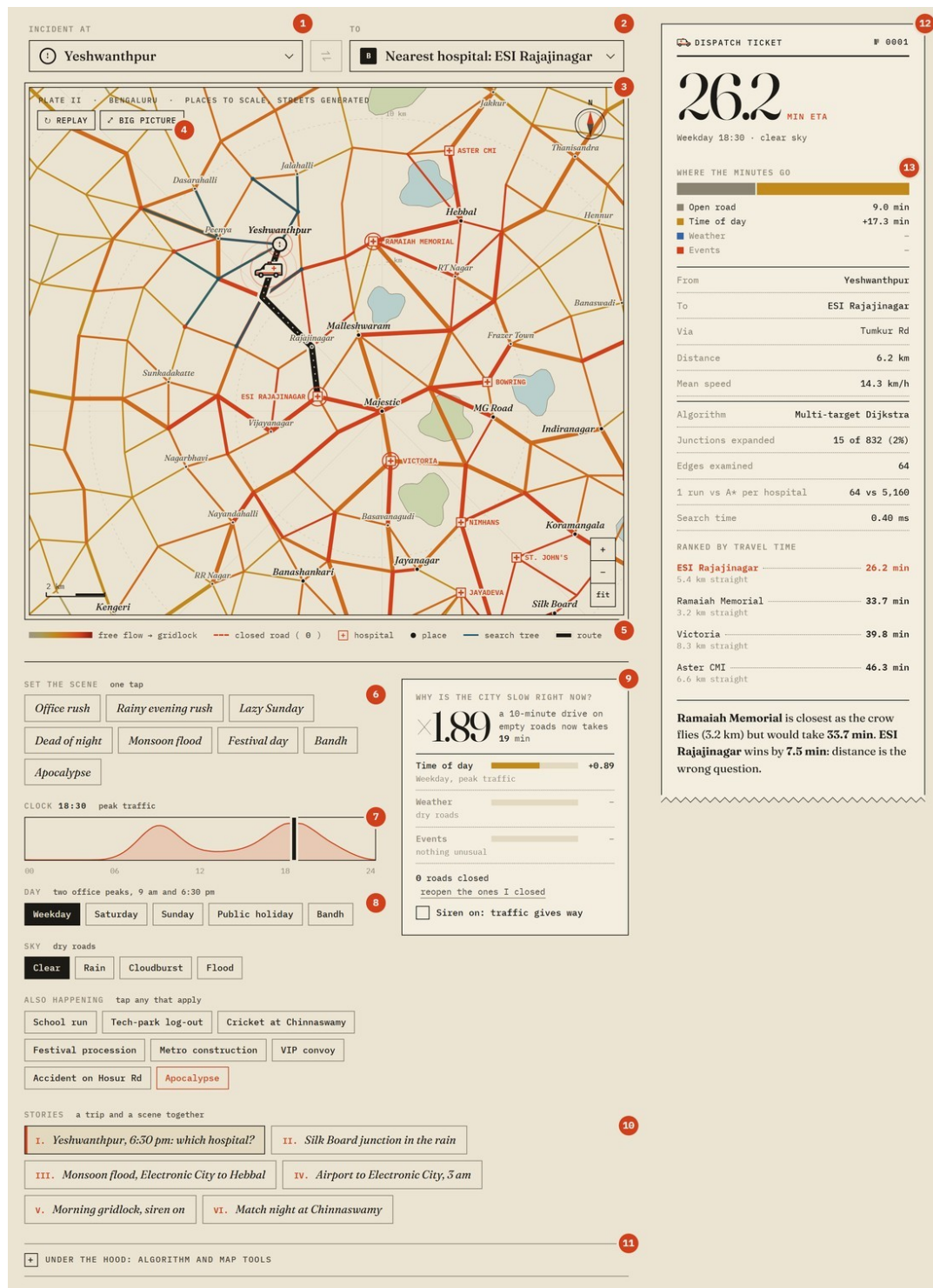


Figure 6. Anatomy of the console. The numbered parts are described in the table below.

NO.	PART	WHAT IT DOES
1	From / Incident at	Searchable list of places for the origin. The label and marker change to "Incident at" in dispatch mode.
2	To	Destination. "Nearest hospital" switches to dispatch; choosing a place switches to an A-to-B trip.
3	Map	Roads coloured by congestion, hospitals, places, the search tree, the route and the ambulance.
4	Replay, Big picture	Re-run the search animation, or open it full screen for step-by-step study.
5	Legend	Colour ramp from free flow to gridlock, closed roads (with a live count), symbols.

NO.	PART	WHAT IT DOES
6	Set the scene	Eight one-tap presets from "Dead of night" to "Apocalypse".
7	Clock	Time of day in 15-minute steps, drawn over the traffic curve for the chosen day type.
8	Day, Sky, Also happening	Five day types, four skies, and eight events that can be combined.
9	Why is the city slow?	Average slowdown on every road, split into time of day, weather and events; closed-road count; siren.
10	Stories	Six prepared cases, each a trip and a scene together.
11	Under the hood	Collapsed by default: the algorithm selector and the click tool (place a point, or close a road).
12	Dispatch ticket	ETA, route details, search statistics, hospitals ranked by travel time, and a one-line verdict.
13	Where the minutes go	The ETA of this route split into open road, time of day, weather and events.

5.2.1 Place pickers

Origin and destination are chosen from 62 places and 15 hospitals. The list is grouped into seven zones and shows each place's distance and direction from Majestic. Typing filters the list, with names that start with the text ranked first; arrow keys and Enter work without a mouse. The destination list begins with "Nearest hospital", so one control covers both kinds of question.

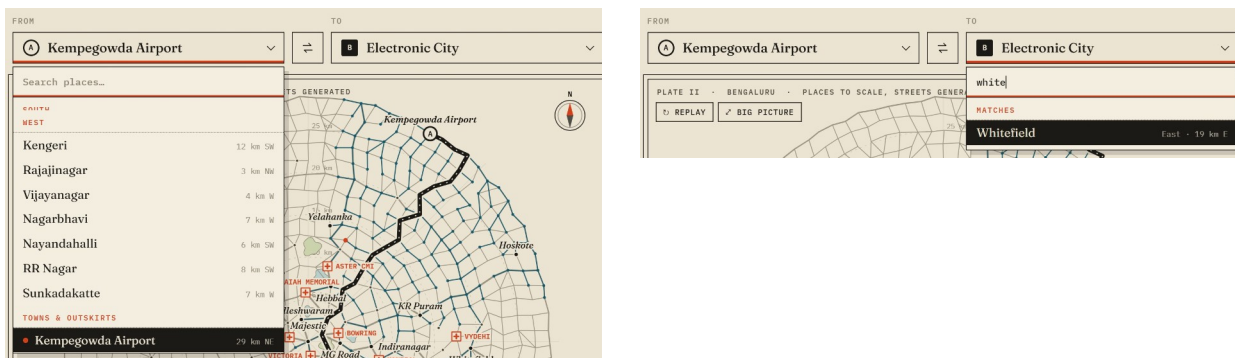


Figure 7. Place pickers: the grouped list for the origin (left) and a search for "white" in the destination list (right).

5.2.2 The map

The map is drawn on a canvas and redrawn every frame while anything is moving. A search is replayed in the order the algorithm visited junctions, so the teal tree spreads exactly as the search did. When it reaches the goal the route is inked in black on a paper-coloured casing, the one colour the traffic ramp never uses, and an ambulance drives it. The camera glides to frame each new trip, so a short trip zooms in and reveals smaller localities.

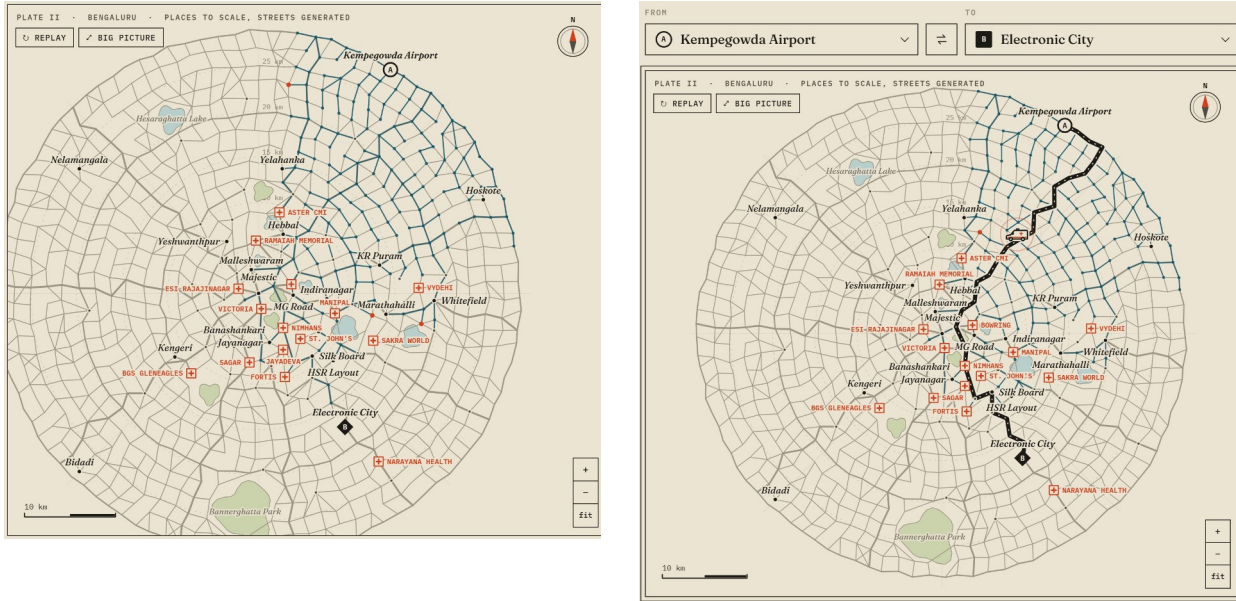


Figure 8. *A** from the airport to Electronic City at 3 am: the search tree mid-flight (left), and the finished route with the ambulance on it (right).

LAYER (BOTTOM TO TOP)	CONTENT
Survey grid	Rings every 5 km and eight compass bearings from the city centre.
Water and parks	Eleven lakes and parks; no roads cross them.
Roads	Width by class (local, ring, highway); colour by congestion in six steps.
Closures	Dashed red with a cross; closed by the scene or by the user.
Search tree	Teal, in visit order; the second front of bidirectional search in brown; the newest visits in red.
Route and ambulance	Ink on a paper casing; the ambulance faces its direction of travel.
Hospitals and places	Labels are placed by priority and skipped if they would overlap; minor places appear when zoomed in.
Furniture	Scale bar, compass, zoom buttons, and a tooltip naming the road under the pointer with its current time.

5.2.3 Scene builder and "Why is the city slow?"

The scene is set in plain language rather than with technical controls. Each road's time is its free-flow time multiplied by one plus three delay terms, so every cause of delay can be shown separately:

$$\text{time on a road} = \text{free-flow time} \times (1 + k \times (\text{time of day} + \text{weather} + \text{events}))$$

Here k is 1 normally and 0.45 with the siren on. Every term is zero or positive, so no scene can make a road faster than its free-flow time. That property is what keeps the A^* heuristic valid in every scene (section 6.2).

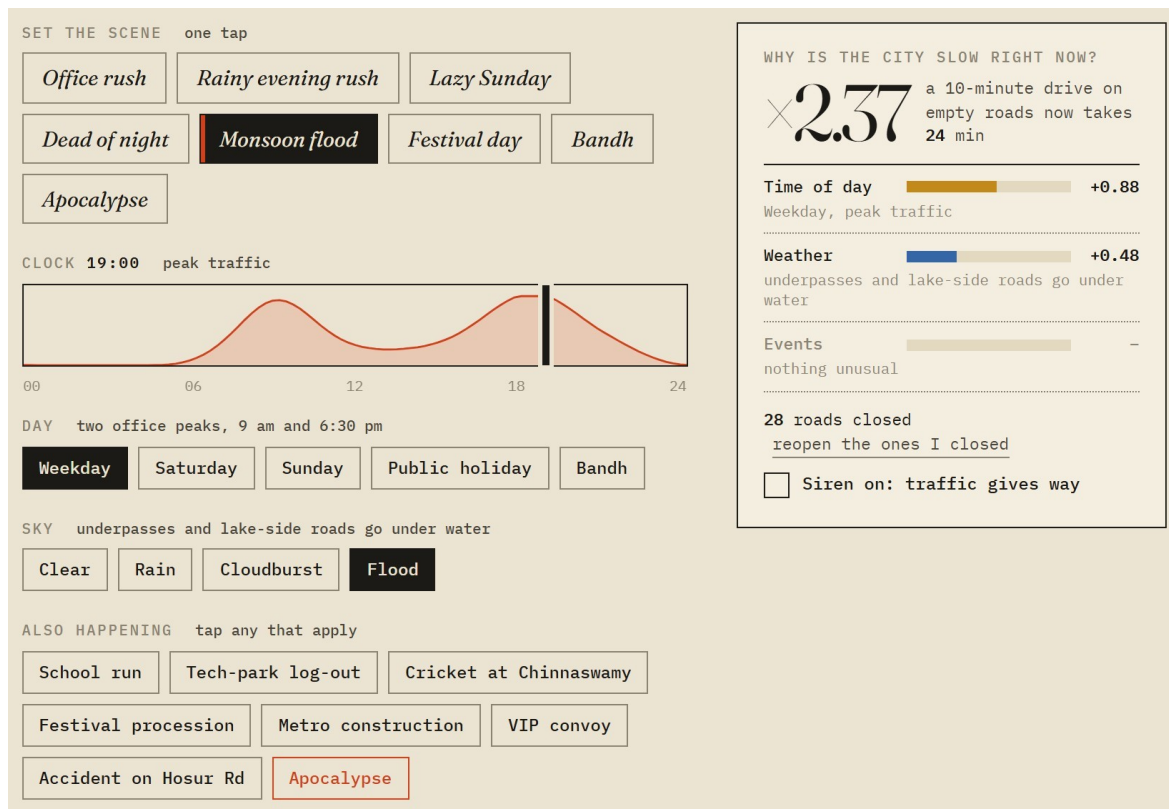


Figure 9. The scene builder set to a monsoon flood at 19:00. The panel on the right explains the slowdown and counts the roads the flood has closed.

CONTROL	OPTIONS	EFFECT ON THE MODEL
Day	Weekday, Saturday, Sunday, Public holiday, Bandh	Chooses the daily traffic curve. A weekday has two office peaks; a bandh leaves roads nearly empty.
Clock	00:00 to 24:00 in 15-minute steps	Position on that curve. Delay is heavier in the core and at five chronic hotspots such as Silk Board.
Sky	Clear, Rain, Cloudburst, Flood	Slows every road, most near lakes and known waterlogging spots. A flood also closes the lowest-lying roads.
Also happening	School run, Tech-park log-out, Cricket at Chinnaswamy, Festival procession, Metro construction, VIP convoy, Accident on Hosur Rd, Apocalypse	Adds local jams around the places involved; some events close roads.
Siren	On or off	Multiplies all three delay terms by 0.45 for the ambulance.
Road closure	Shift-click any road	Sets that road's time to infinity; every algorithm skips it.

5.2.4 Dispatch ticket

The ticket is the output of the routing problem in readable form. It also reports how much work the search did, and compares that with the obvious alternative: Dijkstra for an A-to-B trip, or one A* run per hospital for a dispatch.

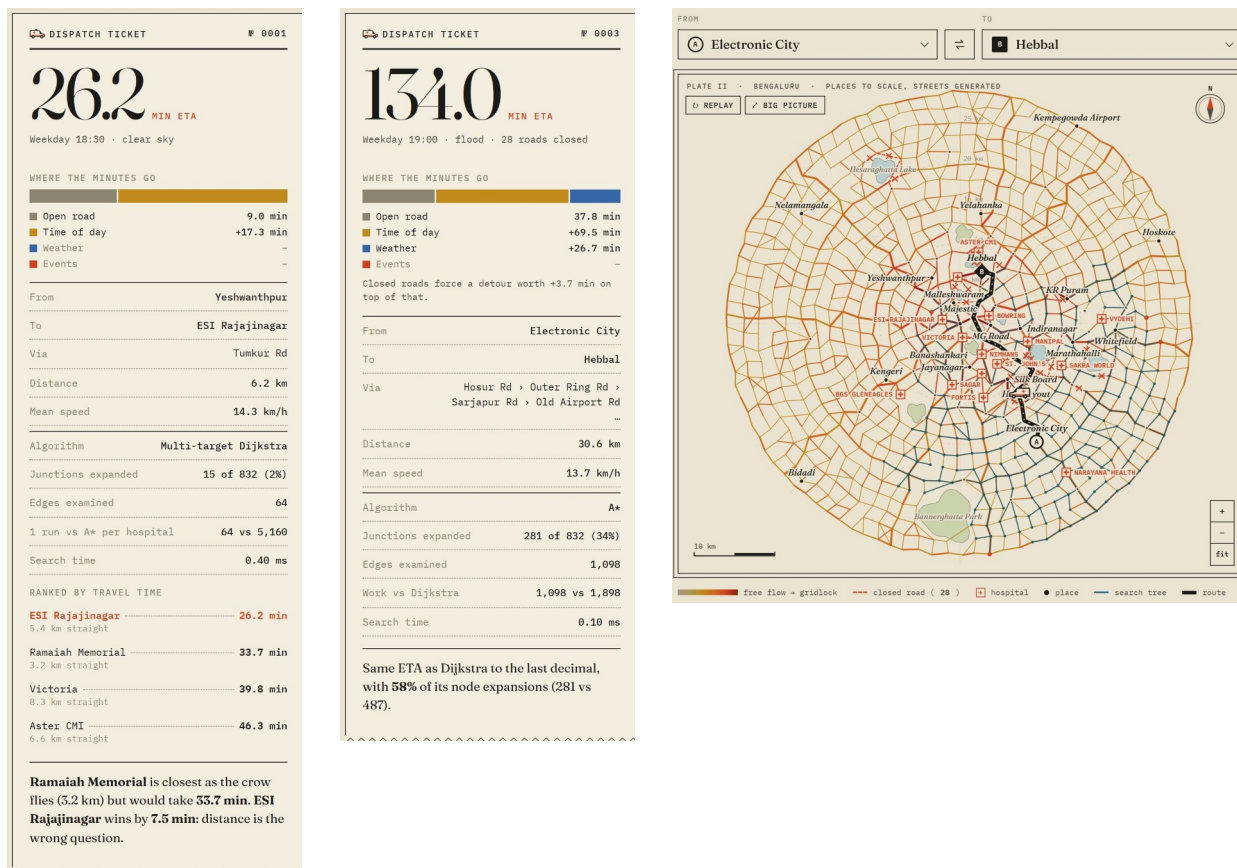


Figure 10. Left: a dispatch from Yeshwanthpur, where the closest hospital is not the fastest. Centre and right: Electronic City to Hebbal in a flood, with the ETA split by cause and the closed roads on the map.

5.2.5 Under the hood

The algorithm selector and the click tool are kept out of the way, because most viewers only need the pickers and the scene. Opening this panel lets an examiner run the same trip with each of the five algorithms and watch the ticket's statistics change.

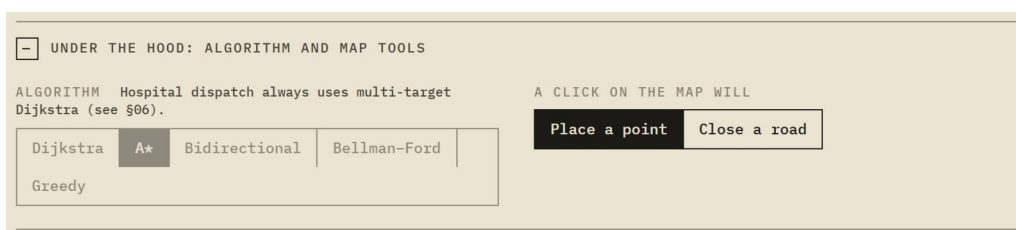


Figure 11. The collapsed panel opened: algorithm selector and click tool.

5.3 The race

All five algorithms answer the same trip on the same traffic. The shared clock counts **edges examined**, the unit of real work, so the result does not depend on how fast the machine is. In the run captured below (Kempegowda Airport to Electronic City), the four optimal algorithms returned the same ETA and differed only in effort; greedy best-first finished first and returned a slower route.

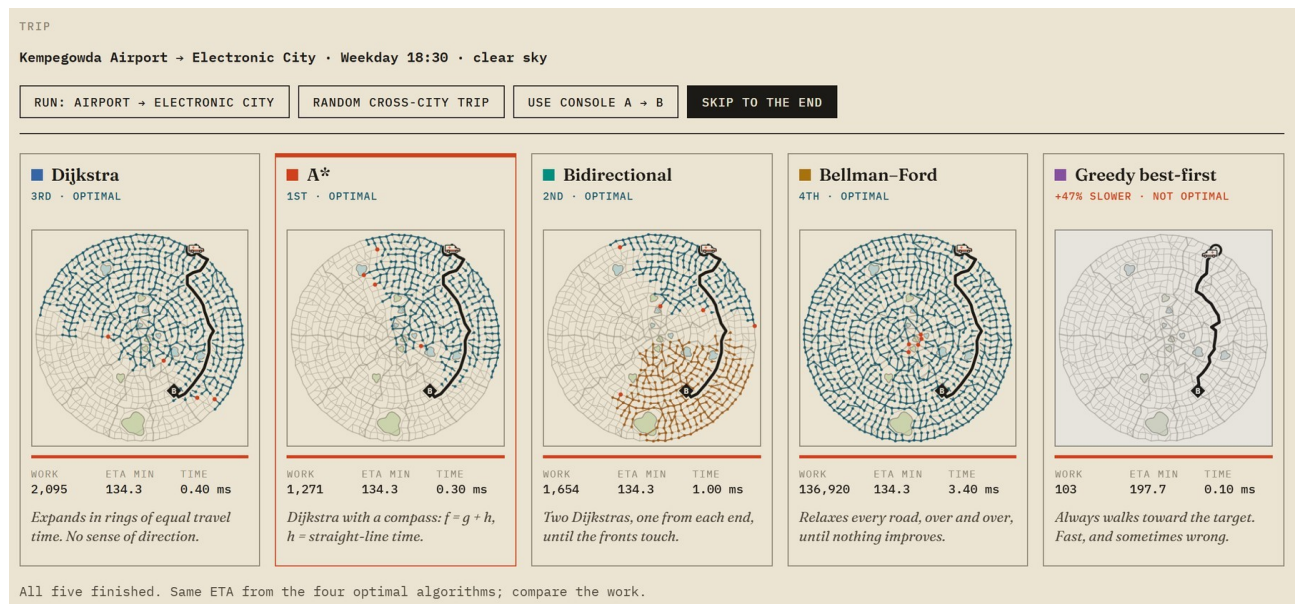


Figure 12. The race after all five have finished. A* examined the fewest edges among the algorithms that found the optimum.

ALGORITHM	RESULT	EDGES EXAMINED	ETA (MIN)	SEARCH TIME
Dijkstra	3rd, optimal	2,095	134.3	0.40 ms
A*	1st, optimal	1,271	134.3	0.30 ms
Bidirectional	2nd, optimal	1,654	134.3	1.00 ms
Bellman-Ford	4th, optimal	136,920	134.3	3.40 ms
Greedy best-first	+47% slower, not optimal	103	197.7	0.10 ms

5.4 The big picture

Any search can be opened full screen. The viewer can pause, drag the scrubber to any step, change speed from one quarter to ten times, and switch algorithm on the same trip. A small curve on the right plots edges examined against junctions visited, with a cursor that follows the replay. Any of the five race maps can also be clicked to open it here.

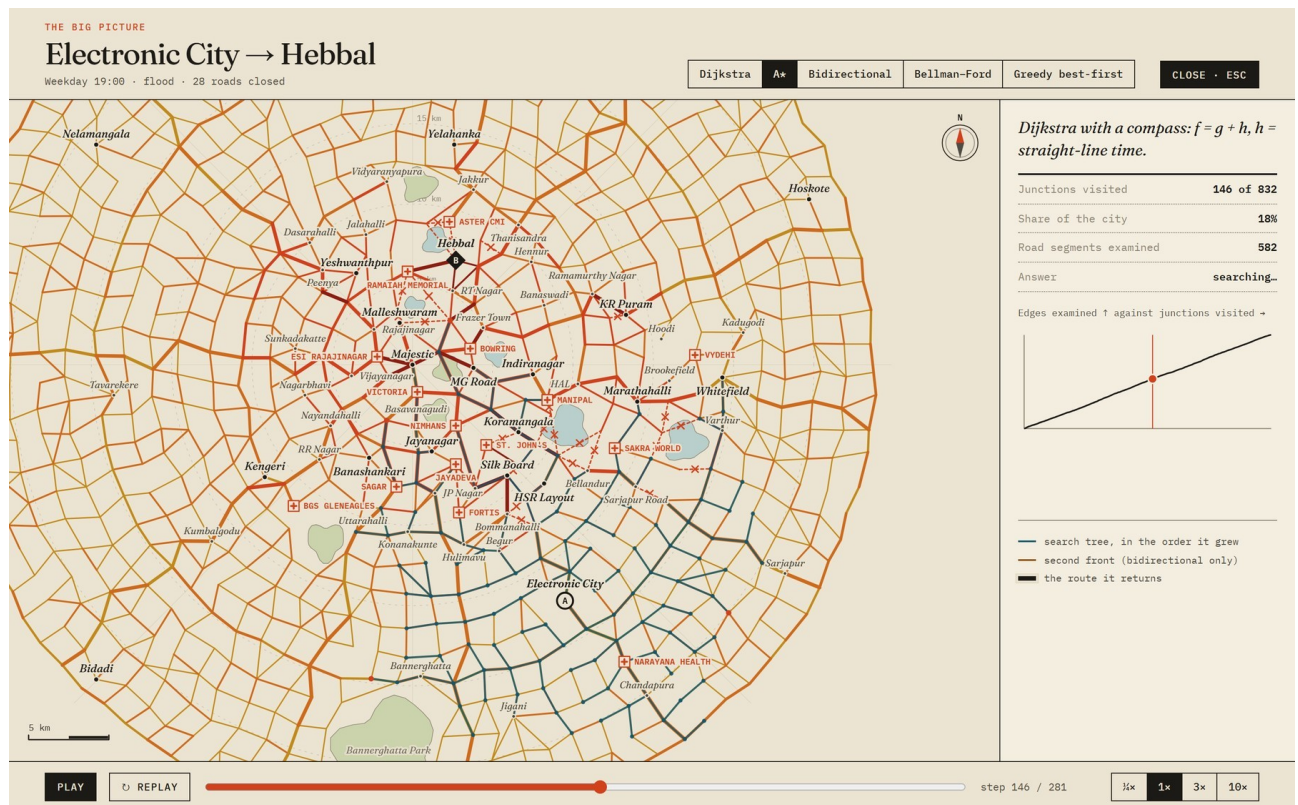


Figure 13. The big picture paused part-way through an A* search in a flood. The counters on the right are live.

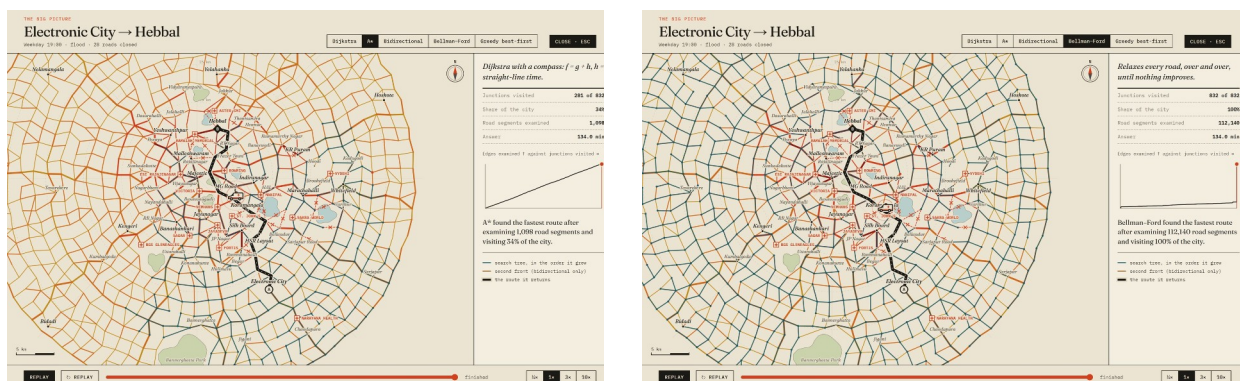


Figure 14. The same trip finished. Left: A* visited 34% of the city. Right: Bellman–Ford visits every junction and keeps working after it has seen them all.

5.5 The proof ledger

Ten claims about the implementation are tested each time the section scrolls into view. Nothing is hard-coded: each test builds its own city, runs the real algorithms and compares them with a hand-worked answer or with each other. In the captured run the summary read "10 / 10 passed in 1.2 s".

10 / 10 PASSED IN 1.2 S

RUN ALL TESTS AGAIN

VERIFIED 10/10

T1	Hand-checked graph On a 7-node graph whose answer was worked out on paper (cost 5 via the detour 0-1-4-5-2-3), every optimal algorithm agrees. <u>dijkstra 5 · astar 5 · bidir 5 · bellman 5 · greedy 7</u>	✓ PASS	3 ms
T2	Source equals target Routing from a junction to itself costs 0 and returns a one-node path, for every algorithm. <u>5 of 5 algorithms return cost 0, path [5]</u>	✓ PASS	8 ms
T3	Unreachable destination An island node (hand graph) and a junction sealed in by closures (city graph) are reported as unreachable, never as a fake route. <u>island: 0/5 found · sealed junction: 0/5 found (want 0 and 0)</u>	✓ PASS	5 ms
T4	Road closure forces a detour Flood a road on the optimal route: the new route avoids it, costs no less, and all optimal algorithms still agree. <u>25 closures tested · 0 violations · last detour added 14.0 min</u>	✓ PASS	40 ms
T5	Cross-validation, 150 random trips Dijkstra, A*, Bidirectional Dijkstra and Bellman-Ford return the same optimal cost ($\pm 1e-6$ min) on 150 random origin-destination pairs at evening peak. <u>150 pairs $\times$ 4 algorithms = 600 searches · 0 disagreements</u>	✓ PASS	82 ms
T6	The heuristic never lies A* is only correct if h is admissible ($h \leq \text{true cost}$) and consistent ($h(u) \leq w(u,v) + h(v)$). Both are checked on every junction and every road, at three times of day. <u>9,234 inequalities checked · 0 violations</u>	✓ PASS	7 ms
T7	Optimal substructure Every prefix of an optimal route is itself an optimal route to its endpoint (the property Dijkstra and A* are built on). <u>241 prefixes compared with single-source Dijkstra · 0 mismatches</u>	✓ PASS	9 ms
T8	Greedy is fast, and sometimes wrong Greedy best-first never beats the optimum (it cannot), but it should lose on a noticeable share of trips: the reason A* adds g to h. <u>greedy was slower on 131 of 150 trips (avg +29% ETA) · beat the optimum 0 times</u>	✓ PASS	8 ms
T9	Hospital dispatch = best of all hospitals One multi-target Dijkstra run returns the same fastest hospital as running a separate Dijkstra to each hospital and taking the minimum. <u>40 incidents · 15 hospitals · 0 mismatches</u>	✓ PASS	53 ms
T10	Monotone world The siren can never make a trip slower, and free-flow (3 am) can never be slower than rush hour: weights only move one way, so costs must too. <u>40 trips: night $\leq$ siren $\leq$ rush held 40 times</u>	✓ PASS	22 ms

Figure 15. The proof ledger after a run: each claim, its evidence and the time it took.

5.6 The benchmark lab

The lab runs two experiments in the browser and draws the results as interactive charts with no charting library. Hovering shows a crosshair listing every algorithm at that point; clicking a name hides its line; a switch changes between a log and a linear scale. The results can be copied as CSV.

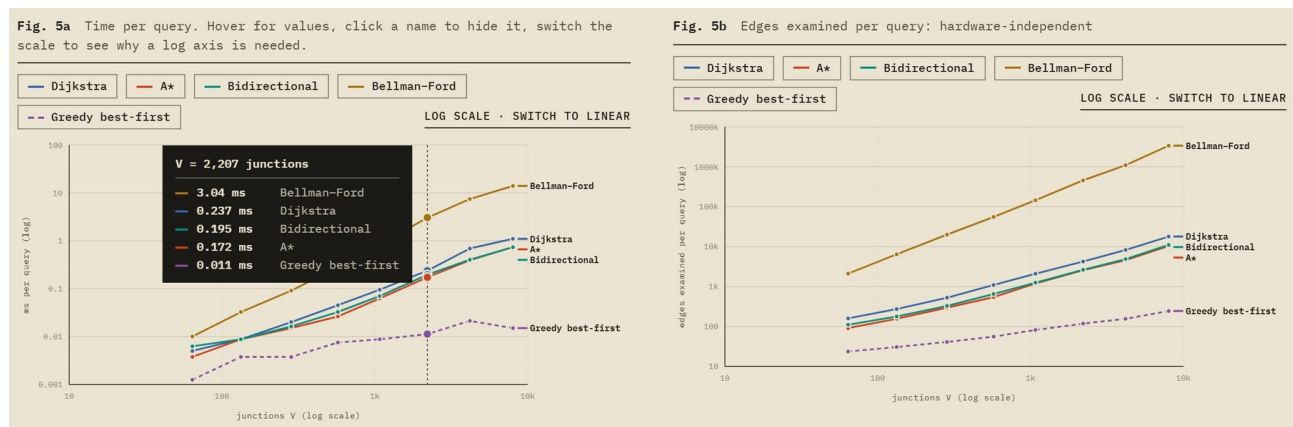


Figure 16. Experiment A: time per query and edges examined per query on eight city sizes, on log scales. A straight line on these axes is a power law.

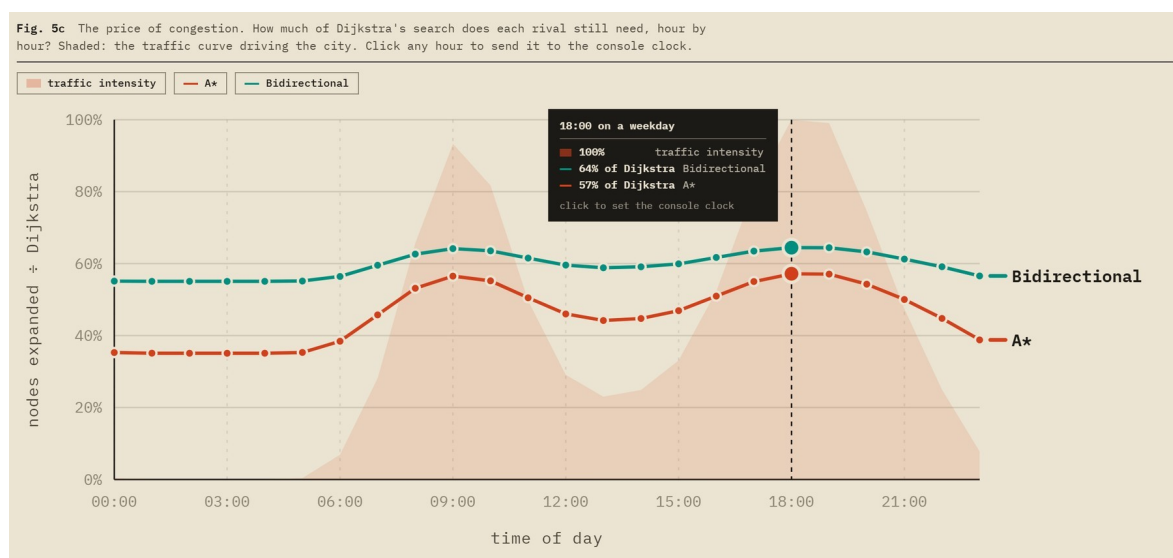


Figure 17. Experiment B: the share of Dijkstra's search that A* and bidirectional search still need, hour by hour. Clicking an hour sends it to the console clock.

ALGORITHM	MS / QUERY	NODES EXPANDED	EDGES EXAMINED	WORK VS DIJKSTRA	MEASURED GROWTH	THEORY
Dijkstra	1.10	4,548	17,860	100%	$V^{0.98}$	$O((V+E) \log V)$
A*	0.741	2,624	10,345	58%	$V^{0.99}$	$O((V+E) \log V)$ worst
Bidirectional	0.733	2,803	11,008	62%	$V^{0.96}$	$O((V+E) \log V)$
Bellman-Ford	14.0	8,011	3,379,678	189×	$V^{1.52}$	$O(V \cdot E)$
Greedy best-first	0.015	60.5	243	1%	$V^{0.48}$	$O((V+E) \log V)$

Largest city: $V = 8,011$ junctions, $E = 15,666$ road segments · averaged over 80 random trips (Bellman-Ford: 15) at 18:30.

Figure 18. The results table for the largest city, with the growth exponent fitted from the data beside the theoretical bound.

5.7 Verdict section

The last content section of the page states the choice and the reasons, and is honest about where A* stops being the best option. It carries the comparison table below; the figures in its last column are filled in from the benchmark the viewer has just run.

ALGORITHM	TIME	SPACE	OPTIMAL?	NEEDS	MEASURED WORK VS DIJKSTRA
■ Dijkstra	$O((V+E) \log V)$	$O(V+E)$	Yes	non-negative weights	100%
■ A*	$O((V+E) \log V)$ worst case; far less with a good h	$O(V+E)$	Yes, if h is admissible	coordinates, admissible h	58% of the edges
■ Bidirectional	$O((V+E) \log V)$	$O(V+E)$	Yes	reversible graph	61% of the edges
■ Bellman-Ford	$O(V \cdot E)$	$O(V+E)$	Yes	nothing (handles negatives)	189x the edges
■ Greedy best-first	$O((V+E) \log V)$	$O(V+E)$	No	coordinates	tiny, but wrong on many trips (test T8)

Figure 19. The comparison table in the verdict section.

5.8 Themes and small screens

A night chart re-inks the whole page, maps included. On a phone the layout becomes a single column, the map keeps its controls, and nothing scrolls sideways.

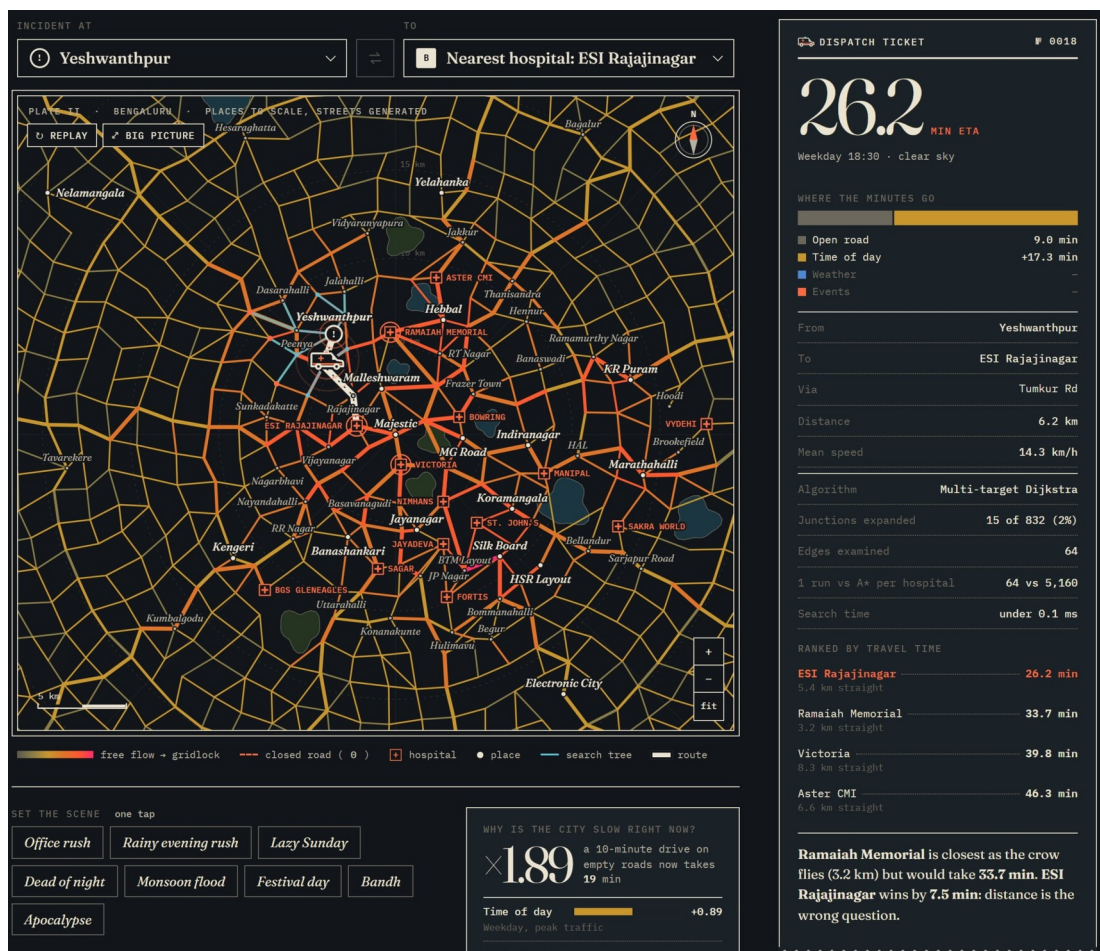


Figure 20. The console in the night chart.

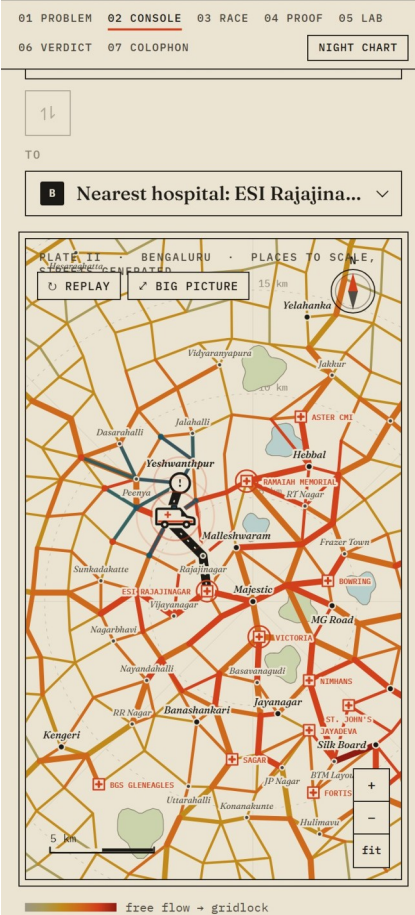
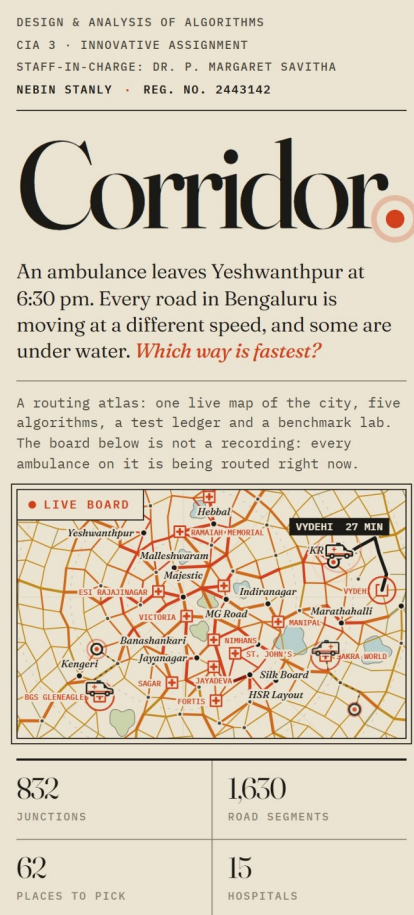


Figure 21. The masthead and the console at phone width (390 px).

6 Algorithms and design justification

6.1 The five algorithms

ALGORITHM	HOW IT SEARCHES	TIME	SPACE	OPTIMAL?
Dijkstra	Settles junctions in order of travel time from the origin, spreading in every direction.	$O((V+E) \log V)$	$O(V+E)$	Yes
A*	Dijkstra ordered by time so far plus an estimate of time to go, so it leans toward the goal.	$O((V+E) \log V)$ worst case	$O(V+E)$	Yes, if the estimate never over-estimates
Bidirectional Dijkstra	Two searches, from the origin and from the goal, that stop when their fronts meet.	$O((V+E) \log V)$	$O(V+E)$	Yes
Bellman–Ford	Re-examines every road, pass after pass, until no time improves.	$O(V \cdot E)$	$O(V+E)$	Yes
Greedy best-first	Always moves to the junction that looks closest to the goal, ignoring time so far.	$O((V+E) \log V)$	$O(V+E)$	No

6.2 Why A* for a trip from A to B

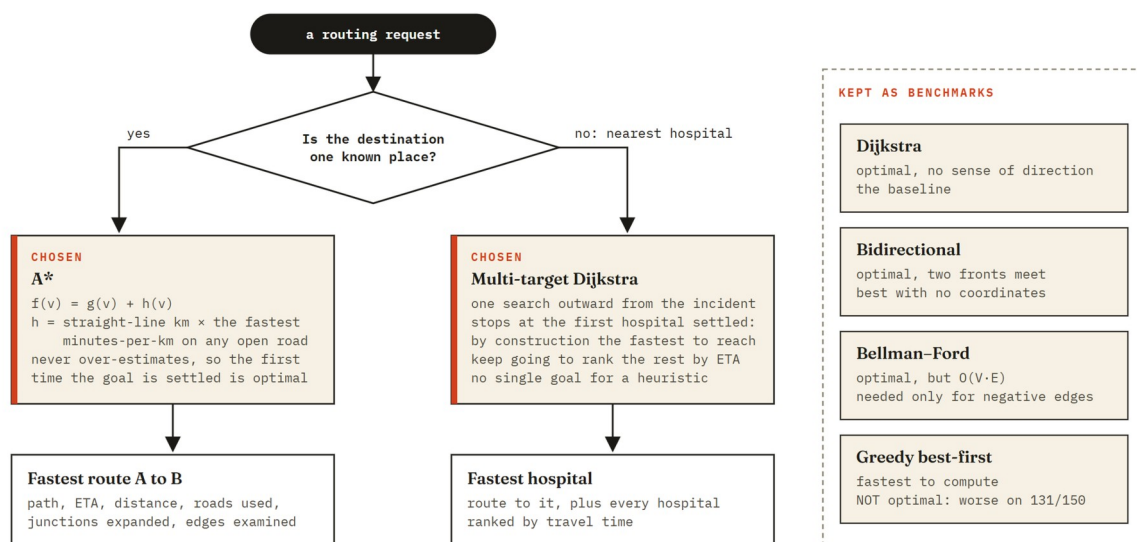


Figure 22. How the prototype chooses an algorithm, and what the others are kept for.

- **The weights suit it.** Travel times are never negative, and a road can never beat the straight line at the network's best speed, so a safe estimate exists.
- **Nothing has to be precomputed.** Faster schemes such as contraction hierarchies prepare shortcuts for fixed weights. Here weights change with the clock and roads close without warning; A* just reads the new numbers.
- **It does less work for the same answer.** On cities of 1,000 junctions or more, A* expanded 57% of the junctions Dijkstra did (section 8.4).
- **The estimate adapts.** $h(v) = (\text{the smallest minutes-per-kilometre on any open road right now}) \times (\text{straight-line distance from } v \text{ to the goal})$. It tightens when the city is fast and loosens when it jams.

The estimate is **admissible**: every road costs at least that minimum rate per kilometre and is at least as long as the straight line, so h never over-estimates. It is also **consistent**: $h(u) \leq w(u,v) + h(v)$ for every road. Consistency means a settled junction never needs to be reopened, and the first time the goal is settled the answer is optimal. Test T6 checks both inequalities on every junction and every road at three times of day.

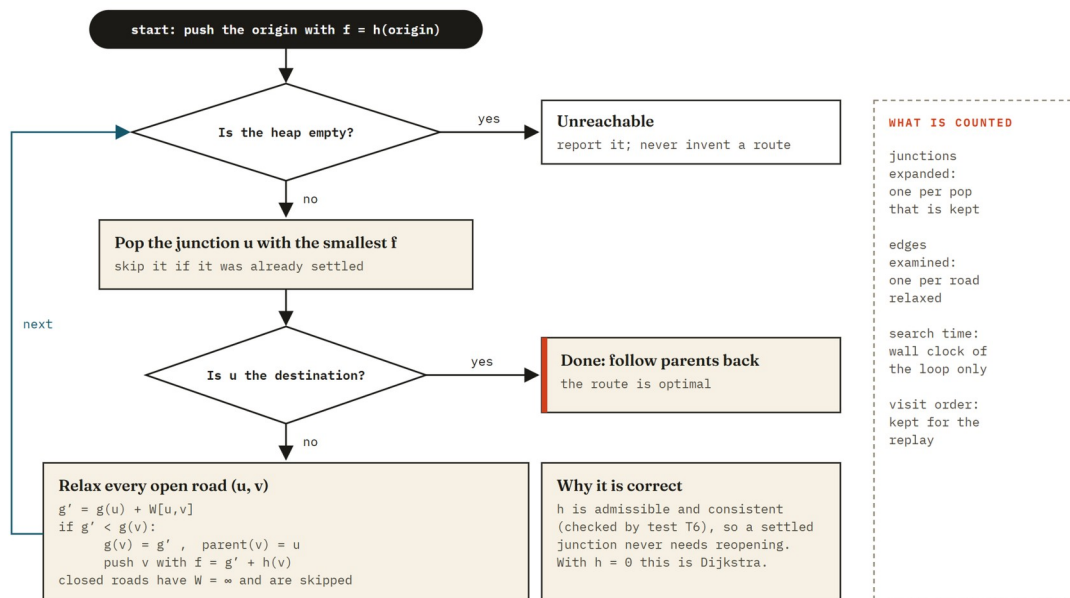


Figure 23. The A* loop as implemented. With $h = 0$ the same loop is Dijkstra's algorithm.

6.3 Why multi-target Dijkstra for dispatch

"Nearest hospital" has no single goal for an estimate to point at. One Dijkstra search from the incident settles junctions in order of travel time, so the first hospital it settles is the fastest to reach, and continuing ranks the rest. In the opening case this single search examined 64 edges, against 5,160 for running A* once per hospital. Test T9 confirms it returns the same hospital as the exhaustive method.

6.4 Complexity, argued

- **Time.** Each junction is settled once and each road is examined at most twice, once from each end. An examination may push to a binary heap at a cost of $O(\log V)$. The total is $O((V+E) \log V)$. With $h = 0$, A* is Dijkstra, so its worst case is the same.
- **Space.** Distance, parent and settled arrays take $O(V)$; the graph takes $O(V+E)$; the heap holds at most one entry per successful examination, $O(E)$.
- **Bellman–Ford.** Up to $V - 1$ passes over all E roads gives $O(V \cdot E)$. In practice it stops after about as many passes as the graph is wide in hops.

6.5 Where A* stops being the answer

A*'s advantage is not constant. It expanded 35% of Dijkstra's junctions at 02:00 but 57% at 18:00: when every road is slow, a straight-line estimate is loose. Fewer expansions also do not convert one-for-one into milliseconds, because each push evaluates a square root. Bidirectional search is a close rival and the better choice on a network with no coordinates. For a static network of millions of junctions, methods that precompute would win outright.

7 Non-functional requirements

ID	REQUIREMENT	EVIDENCE
NFR-01	Performance: a route is found fast enough to re-plan while the clock is dragged.	Search time on the ticket for the opening case: under 1 ms. A cross-city A* search examined 877 edges.

ID	REQUIREMENT	EVIDENCE
NFR-02	Correctness is demonstrable, not asserted.	10 of 10 tests pass in the browser (10 / 10 passed in 1.2 s); 120 benchmark trips cross-checked with 0 disagreements.
NFR-03	Portability: runs without installation.	Plain HTML, CSS and JavaScript. No build step. Opens from disk or any static host.
NFR-04	No third-party code.	No frameworks, chart libraries or map services. Two web fonts, with system fallbacks.
NFR-05	Small footprint.	All eleven source files total 237 kB uncompressed.
NFR-06	Responsive layout.	Checked at 390 px and 1,440 px. Horizontal overflow at phone width: 0 px.
NFR-07	Accessibility basics.	Pickers and charts work from the keyboard; chart colours pass a colour-blind separation check in both themes; text labels never rely on colour alone.
NFR-08	Robustness.	The scripted walk through every module for this document produced 0 page errors.
NFR-09	Privacy.	No data is collected or sent anywhere. All computation is local.
NFR-10	Maintainability.	The engine has no dependency on the page and can be run and tested in Node.

8 Verification and results

8.1 Test cases

ID	TEST	EVIDENCE FROM THE RUN	RESULT
T1	Hand-checked graph	dijkstra 5 · astar 5 · bidir 5 · bellman 5 · greedy 7	Pass
T2	Source equals target	5 of 5 algorithms return cost 0, path [5]	Pass
T3	Unreachable destination	island: 0/5 found · sealed junction: 0/5 found (want 0 and 0)	Pass
T4	Road closure forces a detour	25 closures tested · 0 violations · last detour added 14.0 min	Pass
T5	Cross-validation, 150 random trips	150 pairs x 4 algorithms = 600 searches · 0 disagreements	Pass
T6	The heuristic never lies	9,234 inequalities checked · 0 violations	Pass
T7	Optimal substructure	241 prefixes compared with single-source Dijkstra · 0 mismatches	Pass
T8	Greedy is fast, and sometimes wrong	greedy was slower on 131 of 150 trips (avg +29% ETA) · beat the optimum 0 times	Pass
T9	Hospital dispatch = best of all hospitals	40 incidents · 15 hospitals · 0 mismatches	Pass
T10	Monotone world	40 trips: night ≤ siren ≤ rush held 40 times	Pass

The tests cover the normal case (T1, T5), edge cases (T2 source equals target, T3 unreachable), behaviour under change (T4 closures, T10 scene changes), the theory the algorithms rely on (T6 heuristic, T7 optimal substructure), a deliberate negative result (T8: greedy is wrong on most trips) and the dispatch method (T9).

8.2 Stories

STORY	TRIP	ETA (MIN)	DISTANCE	JUNCTIONS EXPANDED	ROADS USED
I. Yeshwanthpur, 6:30 pm	Yeshwanthpur to ESI Rajajinagar	26.2	6.2 km	15 of 832	Tumkur Rd

STORY	TRIP	ETA (MIN)	DISTANCE	JUNCTIONS EXPANDED	ROADS USED
II. Silk Board in the rain	Silk Board to Fortis	22.1	4.1 km	8 of 832	Hosur Rd
III. Monsoon flood	Electronic City to Hebbal	134.0	30.6 km	281 of 832	Hosur Rd, Outer Ring Rd, Sarjapur Rd, Old Airport Rd ...
IV. Airport to Electronic City, 3 am	Kempegowda Airport to Electronic City	65.8	59.1 km	217 of 832	Satellite Town Ring Rd, Hennur Rd, Old Airport Rd, Hosur Rd
V. Morning gridlock, siren on	Marathahalli to Majestic	36.8	17.7 km	34 of 832	Old Airport Rd, Old Madras Rd
VI. Match night	MG Road to Whitefield	60.2	19.1 km	32 of 832	Old Airport Rd

Story I shows why distance is the wrong measure. Ramaiah Memorial is closest as the crow flies (3.2 km) but would take 33.7 min. ESI Rajajinagar wins by 7.5 min: distance is the wrong question.

8.3 Scenes

SCENE PRESET	AVERAGE SLOWDOWN ON EVERY ROAD	ROADS CLOSED BY THE SCENE
Office rush	× 1.93	0
Rainy evening rush	× 2.07	0
Lazy Sunday	× 1.35	0
Dead of night	× 1.00	0
Monsoon flood	× 2.37	28
Festival day	× 1.38	3
Bandh	× 1.04	0
Apocalypse	× 4.83	123

8.4 Benchmark results

Eight cities were generated, from 64 to 8,011 junctions. On each, the same 80 random trips were run by every algorithm at evening-peak traffic (15 trips for Bellman–Ford on the two largest cities), three times over, keeping the fastest pass. Times vary between machines; the edge counts do not.

JUNCTIONS	ROADS	DIJKSTRA	A*	BIDIRECTIONAL	BELLMAN–FORD	GREEDY
64	130	0.005	0.004	0.006	0.010	0.001
133	259	0.009	0.009	0.009	0.032	0.004
284	560	0.020	0.015	0.016	0.091	0.004
573	1,140	0.045	0.026	0.033	0.280	0.007
1,076	2,075	0.095	0.062	0.070	0.827	0.009
2,207	4,292	0.237	0.172	0.195	3.04	0.011
4,185	8,132	0.688	0.391	0.402	7.43	0.021
8,011	15,666	1.10	0.741	0.733	14.0	0.015

Table: milliseconds per query.

JUNCTIONS	ROADS	DIJKSTRA	A*	BIDIRECTIONAL	BELLMAN-FORD	GREEDY
64	130	159	90	110	2,106	23
133	259	273	155	178	6,404	30
284	560	524	296	327	19,908	41
573	1,140	1,091	542	649	55,347	55
1,076	2,075	2,085	1,183	1,250	144,731	81
2,207	4,292	4,230	2,557	2,623	456,454	117
4,185	8,132	8,245	4,542	4,857	1,096,194	155
8,011	15,666	17,860	10,345	11,008	3,379,678	243

Table: edges examined per query (independent of hardware).

The page writes its own findings from each run. For the run captured in this document they were:

- A* expanded 57% of the junctions Dijkstra did on cities of 1,000 junctions or more; bidirectional search expanded 61%.
- Bellman–Ford examined 189 times as many edges as Dijkstra on the largest city. Its work grew roughly as V to the power 1.5, not V squared: it needs about as many passes as the graph is wide in hops, which grows like the square root of V on a road network.
- Dijkstra's work grew almost exactly in proportion to V , because a random trip floods a roughly constant share of the city.
- All four optimal algorithms returned identical costs on 120 cross-checked trips.
- Greedy best-first examined the fewest edges by far and was slower than the optimum on 131 of 150 test trips (T8).

9 Deployment and operation

ITEM	DETAIL
Live prototype	https://h0gr1d3r-code.github.io/corridor-routing-atlas/
Source repository	https://github.com/H0GR1D3R-CODE/corridor-routing-atlas
Hosting	GitHub Pages, served from the main branch. Any static host works.
Run locally	Open <code>index.html</code> in a browser, or run <code>python -m http.server 5173</code> in the folder and visit <code>http://localhost:5173</code> .
Build step	None.
Folder layout	<code>index.html</code> , <code>css/style.css</code> , and nine files in <code>js/</code> (section 3.1).

10 Limitations and future work

- The street mesh between real places is generated. Loading an OpenStreetMap extract would only change the city-building function.
- Speeds and the strengths of weather, events and the siren are assumed values.
- The graph is undirected: no one-way streets, signals or turn penalties.
- Weights are a snapshot at one clock time; a real system would stream live speeds.
- Only one ambulance is routed at a time. Assigning a fleet to several incidents is a different problem.
- Next step for speed: landmark-based estimates (ALT) or customisable contraction hierarchies, which keep optimality while surviving weight changes.

11 Traceability to the assignment rubric

CRITERION	MARKS	IN THE PROTOTYPE	IN THIS DOCUMENT
Explains the routing problem, inputs, outputs and chosen algorithm	4	Section 01, "The problem"	§1.2, §6
Prototype works and demonstrates the solution with suitable test cases	5	Section 02 console; section 04 proof ledger	§5.2, §5.5, §8.1, §8.2
Explains why the algorithm is suitable and compares it with alternatives	3	Section 03 race; section 06 verdict	§5.3, §6.2 to §6.5
Discusses time and space complexity and gives experimental results	4	Section 05 lab; "Complexity, argued"	§6.4, §8.4
Explains the solution through an effective medium	2	The page itself is the interactive explanation	This document; §5
Creativity and meaningful practical application	2	Live board, scene builder, hospital dispatch, big picture	§5.1, §5.2.3, §5.4